

Lecture 8

Software Interrupts

Interrupts

- Interrupts are mechanisms that allow programs to temporarily stop and transfer control to a special procedure (ISR) for handling specific tasks, such as hardware events or system services.
- Used for services such as I/O operations, system calls, and other OS-level tasks.
- Executed using an interrupt instruction, like INT in x86 assembly.

Types of Interrupts

- There are two main types of interrupts:
- **Hardware Interrupts:** Triggered by hardware devices (e.g., keyboard input, I/O devices).
- **Software Interrupts:** Triggered by software instructions to request services (e.g., operating system functions).

Software vs. Hardware Interrupts

Aspect	Software Interrupts	Hardware Interrupts
Trigger Method	Program instruction (INT)	External hardware signals
Usage	System calls, debugging	I/O devices, system clock
Control	Software-controlled	Hardware-generated

For example, to read a file from disk or a character from the keyboard.

For example, whenever a key is pressed, the 8086 must be notified to read a key code from the keyboard buffer

How Software Interrupts Work

- **Execution of INT Instruction:** For example, INT 21h in x86 for DOS services.
- **Interrupt Vector Table (IVT):** CPU looks up the ISR (Interrupt Service Routine) address in the IVT.
 - ISR is a special type of Procedure
- **Switch to ISR:** Control passes to the specified ISR.
- **Return to Program:** After ISR execution, control returns to the original program.

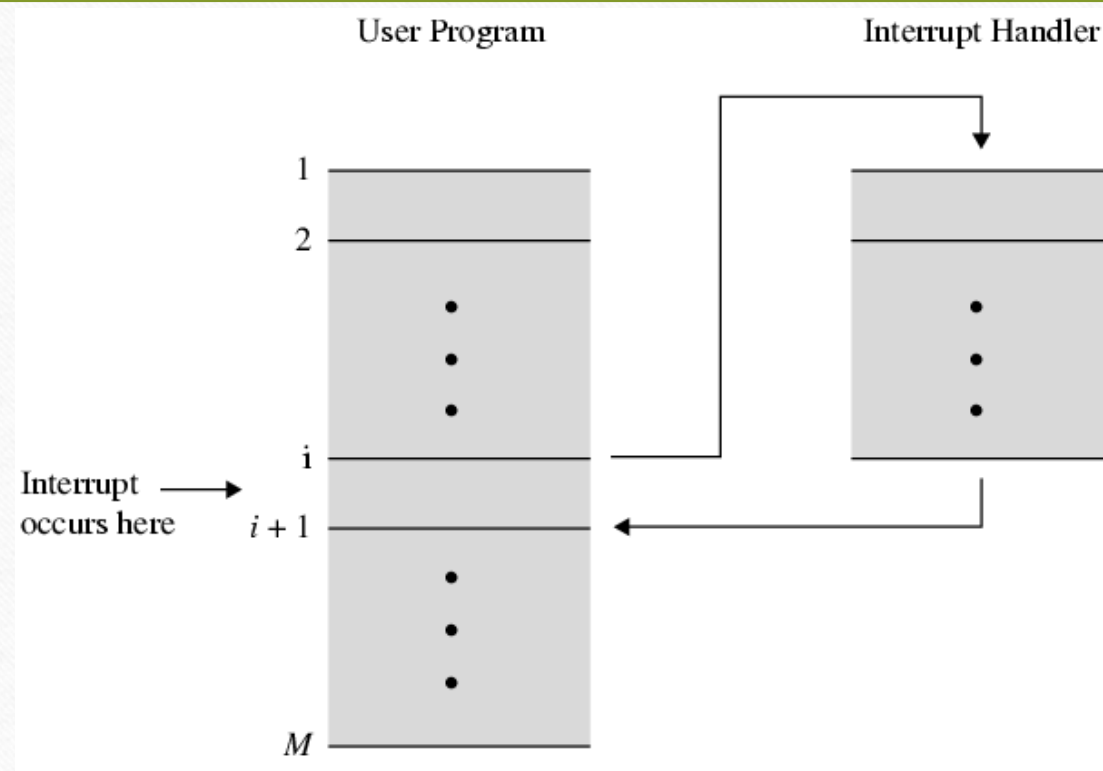
Interrupts – Changing Program Flow

- Mechanism by which other modules (e.g. I/O) may interrupt normal sequence of processing
- Program
 - e.g. overflow, division by zero (Exceptions)
- Timer
 - Generated by internal processor timer
 - Used in Pre-Emptive Multi-Tasking
- I/O
 - From I/O controller
- Hardware failure

Interrupt Cycle

- Added to instruction cycle
- Processor checks for interrupt
 - Indicated by an interrupt signal
- If no interrupt, fetch next instruction
- If interrupt pending:
 - Suspend execution of current program
 - Save context in **stack**
 - Set Program Counter to start address of interrupt handler routine
 - Process interrupt
 - Restore context from **stack** and continue interrupted program

Transfer of Control via Interrupts



Multiple Interrupts

- Disable interrupts
 - Processor will ignore further interrupts whilst processing one interrupt
 - Interrupts remain pending and are checked after first interrupt has been processed
 - Interrupts handled in sequence as they occur
- Define priorities
 - Low priority interrupts can be interrupted by higher priority interrupts
 - When higher priority interrupt has been processed, processor returns to previous interrupt

Interrupt Numbers

- Total 256 interrupts
- 0x00 to 0xFF
 - 0 – 255
- Not all interrupt numbers are used in the same way or assigned to specific services

Interrupt Numbers Categories

- Interrupts 0x00 to 0x1F (0 to 31 in decimal)
 - Reserved for system-level interrupts (e.g. Int 10h)
- Interrupts 0x20 to 0x3F (32 to 63 in decimal)
 - Hardware Interrupts (Int 21h)
- Interrupts 0x40 to 0x7F (64 to 127 in decimal)
 - System-level services
- Interrupts 0x80 to 0xFF (128 to 255 in decimal)
 - Used for software interrupts (Int 80h for system calls)

Common Interrupt Examples

- **INT 10h:** Video services (BIOS interrupt for screen operations).
- **INT 16h:** Keyboard services (BIOS interrupt for reading keyboard input).
- **INT 21h:** DOS services (used for input/output operations, file management, exit program etc.).

Interrupt Vector Table (IVT)

- **Structure:** Array of pointers to ISR addresses.
- **Location:** Typically, in low memory (e.g., 0000:0000 in x86).
 - IVT resides in memory at physical address 0x00000
- Each entry stores the logical address of ISR, which is 4 bytes (2 for segment, 2 for offset).
 - Total size of the IVT is $256 \times 4\text{B} = 1024\text{B}$ (approx. 1 KB)
- To index the IVT, the processor multiplies the interrupt number by 4 to get the physical address of the entry against that interrupt number.

BIOS Interrupts

- The **Basic Input/Output System (BIOS)** is firmware that provides low-level control over computer hardware, including keyboard, display, and disk operations.
- BIOS interrupts are used for direct interaction with hardware components.
- Provides services like video display, keyboard input, print character, and system control.
- Low-level, hardware-focused services like printing on the screen and reading keyboard input.

DOS Interrupts

- **Disk Operating System (DOS)** is a family of operating systems that provide a command-line interface for interacting with the computer.
- DOS interrupts provide software services to manage tasks like file operations, print string, and program termination.
- DOS interrupts are higher-level services compared to BIOS interrupts.
- They offer more abstracted functionality for easier application development.
- Higher-level, system-related services like printing strings and managing files.

BIOS and DOS Interrupts

Interrupt#	Service#	Description	BIOS/DOS
0x21	0x1	read character from standard input, with echo , result is stored in AL . if there is no character in the keyboard buffer, the function waits until any key is pressed.	DOS
0x21	0x7	read character from standard input, without echo , result is stored in AL . if there is no character in the keyboard buffer, the function waits until any key is pressed.	DOS
0x21	0x2	write character to standard output device. entry: DL = Character to write. After execution AL = DL .	DOS
0x21	0x9	Print string at DS: DX on screen. String must be terminated by ' \$ '.	DOS
0x21	0x4C	Terminate program and return control to DOS	DOS
0x10	0x2	Set Cursor position. DH ← ROW# DL ← COL # BH ← Page# (i.e., 0)	BIOS
0x10	0x9	Display character & attributes on current cursor position. AL = character to display . BH = page number (i.e. 0) . BL = Color attributes . CX = number of times to write character .	BIOS

Example 1

```
.model small

.stack 100h

.data
Msg db "Hello$"
.code

Mov ax,@data
Mov ds,ax

Mov ah,0x9           ;display string on screen
Mov dx,offset msg
Int 0x21

Mov ah,0x4c          ;terminating program
Int 0x21
```


Example 2

```
.model small

.stack 100h

.data

n db ?

.code

Mov ax,@data
Mov ds,ax

mov ah,0x1  ; read character from keyboard
int 0x21

sub al,0x30 ; subtract 0x30 from the character to make it a number
mov n,al    ; store the number to variable

mov ah,0x4c
int 0x21
```